

Sequence Integrity Matters: Row-Level Client Partitioning Artifacts in Federated RAN Time-Series Benchmarks

Hsiu-Chi Tsai

National Yang Ming Chiao Tung University
nctuie@gmail.com

Abstract—Federated-learning (FL) benchmarks routinely synthesise client heterogeneity by partitioning a pooled dataset across clients with a Dirichlet distribution over rows. For tabular or image data this is harmless. For *time series*—where each client subsequently builds sliding windows—it is not: row-level partitioning scatters a sequence’s consecutive timesteps across clients, so the per-client windows are no longer real trajectories. We show that this *sequence-integrity* violation can manufacture a striking but spurious finding—that *more* client heterogeneity *improves* accuracy—in a federated O-RAN slice-SLA benchmark (CoO-RAN), and we dismantle it. The fragmentation itself is universal (a deterministic consequence of the partition-then-window recipe, confirmed on two real RAN datasets), but its impact on accuracy is task-conditional: it degrades AUC substantially only when the prediction target is *sequence-essential* (a small order-invariant residual remains for window-aggregate targets). We introduce a cheap, partition-free diagnostic, Δ_{traj} —the AUC an intact sequence model gains from temporal order over an order-shuffled baseline—and show the fragmentation AUC gap is a monotone function of it (Spearman 0.95 across 11 CoO-RAN targets; Spearman 1.00 / Pearson 0.99 in a controlled synthetic study). The artifact is architecture-invariant for order-using models (LSTM, GRU) and absent for order-blind models, confirming it is sequence-specific. We give a corrected partitioning protocol (partition by entity/run; synthesise heterogeneity with *run-level* Dirichlet) and recommend Δ_{traj} as a pre-submission screen. On a second real dataset (Open RAN Commercial Traffic Twinning) the mechanism replicates but the AUC impact does not—as the diagnostic predicts for its persistent targets—serving as a negative control.

I. INTRODUCTION

Client heterogeneity (non-IID data) is the defining difficulty of FL, and benchmarks emulate it by splitting a pooled dataset with a Dirichlet(α) distribution: small $\alpha \rightarrow$ skewed, large $\alpha \rightarrow$ near-IID. This recipe is near-universal [1], [2], [3] and, for i.i.d.-sample data (images, tabular rows), statistically sound.

Time-series FL inherits the recipe but adds a step: each client turns its rows into fixed-length sliding windows for a sequence model. Here the recipe quietly breaks. If rows are assigned to clients independently, a run’s consecutive timesteps are *scattered* across clients; when a client then windows its rows (sorted by time), each “window” is a set of non-consecutive snapshots, and the label attached to it belongs to a temporally mismatched step. The sequence model is trained on temporally incoherent (window, label) pairs.

We encountered this concretely. In a federated O-RAN slice-SLA prediction benchmark on the CoO-RAN/Colosseum

testbed [4], partitioning by Dirichlet over rows produced an *inverted* heterogeneity curve: AUC rose as α fell (more skew $\rightarrow$ better accuracy), the opposite of the FL norm—recent large-scale assessments find heterogeneity *harms* accuracy [5]. The tempting reading—that RAN telemetry has special cell-conditional structure—does not survive controls. A **run-level** Dirichlet (whole runs assigned with the same skew) recovers the natural-partition accuracy and erases the inversion; breaking entity coherence while keeping runs intact costs ~ 0 AUC. The apparent inversion is a **sequence-integrity artifact of how the non-IID partition is constructed**, not a property of the data.

Contributions. (1) We identify and characterise the *sequence-integrity pitfall* of row-level partitioning in time-series FL benchmarks, with a simple *fragmentation audit metric*. (2) We show the AUC impact is **task-conditional** and introduce $\Delta_{\text{seq}}/\Delta_{\text{traj}}$, capacity-matched, partition-free diagnostics that *predict* the gap. (3) We give the mechanism (run-rate vs. trajectory; order vs. consecutiveness) and a careful theory: the diagnostic is monotone, *not* a bound (the natural gap $\leq \Delta_{\text{seq}}$ conjecture is falsified). (4) We establish architecture-invariance (LSTM, GRU) and order-blind sanity controls. (5) We give a corrected protocol and a negative control on a second real RAN dataset.

II. RELATED WORK

(a) The Dirichlet partitioning convention. Simulating non-IID clients by a Dirichlet distribution over labels/rows is the de-facto FL-benchmark standard—codified by NIID-Bench [2] and shipped as the default partitioner in FL frameworks [3]. It is sound for *i.i.d.-sample* data; our point is that time-series FL *inherits* it and then adds per-client windowing, where it silently breaks. The “more heterogeneity helps” reading we dismantle is especially suspect because the established norm is the opposite—heterogeneity *harms* [5]—so an *inverted* curve should itself be a red flag for an artifact.

(b) Windowing leakage in time series. A known pitfall is generating windows *before* the train/test split, leaking future context into training [6]. This is a *different failure mode*: centralized train/test causality (it inflates scores), not federated client partitioning. Ours is within-training, federated-specific, and *degrades*—about per-client window integrity across clients.

TABLE I
FRAGMENTATION AUDIT (FRACTION OF CONTIGUOUS WINDOWS).

dataset	intact	random_split	dirichlet
ColO-RAN	1.000	~ 0	~ 0
Twinning	1.0000	0.0002	0.0031

(b') **Temporal-order ablation (our Δ_{traj} method has precedent).** Shuffling timesteps to probe a model's reliance on temporal order is established—temporal-order verification [7] and segment-shuffle representation learning [8], which notes non-adjacent timesteps can carry strong dependencies. We do *not* claim the shuffle ablation as novel; we reuse it for a new purpose—predicting/screening the federated fragmentation gap—and show (Sec. V) that the shuffle destroys *order* whereas row-level fragmentation destroys *consecutiveness*.

(c) **Temporal fragmentation as a deployment challenge.** Some FL works note local time-series can be “fragmented” and that this degrades feature extraction. We differ: the fragmentation here is *manufactured by the row-level recipe as a benchmark artifact*, is *task-conditional* with a *predictive diagnostic*, and can produce a *qualitatively false finding*. FedSL [9] instead studies legitimately sequentially-split sequences—a different setting.

(d) **FL time-series / RAN benchmarks.** FL traffic/throughput forecasting [10], [11], [12], O-RAN toolchains [13], [4], [14], and FL forecasting benchmarks [15] are the landscape. Susceptibility hinges on the partition-then-window order (Sec. VI): entity-partitioned benchmarks are safe; row-Dirichlet ones are at risk.

III. THE SEQUENCE-INTEGRITY PITFALL

A pooled series is split into clients, then each client builds length- L sliding windows within its rows (sorted by step). *Intact* partitions (natural entity, or run-level Dirichlet) keep whole groups together $\rightarrow$ genuine trajectories. *Row-level* partitions scatter a group's rows $\rightarrow$ fragmented windows.

Audit metric. The fragmentation score = fraction of per-client windows with contiguous `step_idx`: 1.0 for intact, ~ 0 for row-level on both datasets we study.

The audit is *necessary but not sufficient* for an AUC impact—a window can be fragmented yet the task unharmed (Sec. IV). Fragmentation alters within-window *order*, which steps populate a window, and the window-to-label *alignment*.

IV. THE $\Delta_{\text{seq}} / \Delta_{\text{traj}}$ DIAGNOSTIC AND MECHANISM

Diagnostics (capacity-matched, partition-free). $\Delta_{\text{seq}} = \text{AUC}(\text{seq LSTM}) - \text{AUC}(\text{same LSTM, length-1 window})$: value of the multi-step window over a single step. $\Delta_{\text{traj}} = \text{AUC}(\text{seq LSTM}) - \text{AUC}(\text{same LSTM, order-shuffled windows})$: isolates the *order/trajectory* value (the partition-vulnerable part); the cleaner predictor.

Gap decomposition. The gap splits into (i) a *trajectory component* (only sequence models; $\propto \Delta_{\text{traj}}$; dominant for

TABLE II
SOURCE-COLUMN LAG-1 AUTOCORRELATION (COLO-RAN).

source	autocorr	nature	$\Delta_{\text{traj}}/\text{gap}$
tx_brat_dl	0.984	run-rate	~ 0
dl_buffer	0.999	run-rate	~ 0
dl_mcs	0.902	run-rate	~ 0
dl_cqi	0.553	moderate	~ 0
ul_bler	0.022	trajectory	0.22/0.15

TABLE III
COLO-RAN Δ_{seq} LAW (11 TARGETS).

target	Δ_{seq}	Δ_{traj}	gap
buffer_med	0.013	0.000	+0.000
mcs_med	0.039	0.002	+0.000
cqi_med	0.048	0.001	−0.011
brate_med	0.111	0.010	−0.008
bler_trend5	0.159	0.060	+0.083
bler_k2	0.170	0.200	+0.134
bler_k3	0.209	0.219	+0.121
bler_th20	0.221	0.222	+0.149
bler_th10	0.227	0.225	+0.152
bler_th05	0.227	0.225	+0.152
bler_k5	0.233	0.232	+0.158

white-noise targets) and (ii) a small *window-content component* (any model, only for *aggregate* targets). A no-sequence mean-pool MLP isolates (ii): 0.000 for a point target, a real 0.025 for a 5-step-smoothed target ($\ll$ the LSTM's 0.082).

Mechanism: run-rate vs. trajectory. A target's predictability splits into a partition-invariant *run-rate* (estimable order-free from any sample) and a partition-vulnerable *trajectory*. The source's lag-1 autocorrelation is a pre-diagnostic (Table II).

A diagnostic, not a bound. The gap is monotone in Δ_{traj} but we do *not* claim a bound. The conjecture $\text{gap} \leq \Delta_{\text{seq}}$ is *falsified*: with a pure-trajectory synthetic target, fragmented training is mis-led below the single-step ceiling ($\text{gap} > \Delta_{\text{seq}}$), whereas real BLER (with a run-rate fallback) gives $\text{gap} < \Delta_{\text{seq}}$.

V. EXPERIMENTS

5.1 Synthetic causal control. Multivariate AR runs; a knob λ interpolates a target between instantaneous and trajectory dependence; pos-rate held ~ 0.5 , all else fixed. The gap is monotone in Δ_{seq} (Pearson 0.986) and the $\text{gap} \leq \Delta_{\text{seq}}$ bound is falsified at high λ .

5.2 ColO-RAN Δ_{seq} law (main result). 11 targets, 5 seeds, OOD-by-tr split (Table III). The gap is monotone in Δ_{seq} (Pearson 0.943, Spearman 0.918, OLS slope CI95 [0.706, 1.113] excludes 0) and cleaner in Δ_{traj} (Pearson 0.974, Spearman 0.945). BLER targets: gap 0.08–0.16 (5-seed paired-bootstrap CIs exclude 0); CQI/MCS/buffer/throughput: gap ~ 0 . `bler_th10` reproduces the established ColO-RAN gap (intact 0.908 / row 0.757). The law and its Δ_{traj} refinement are shown in Fig. 1.

5.3 Architecture invariance + sanity. A no-sequence mean-pool MLP shows gap ~ 0 even for high- Δ_{seq} BLER

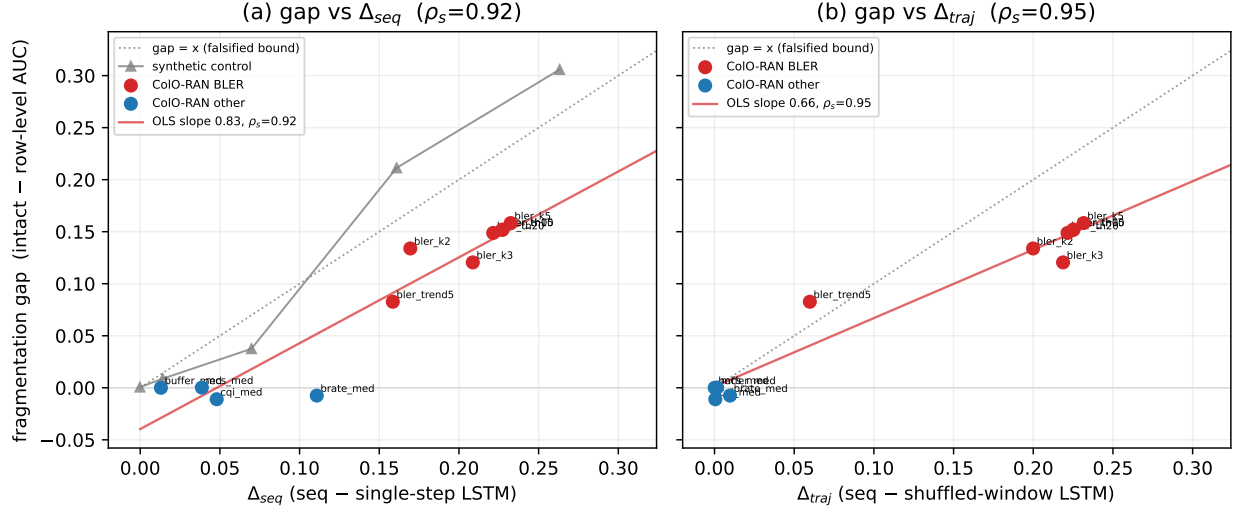


Fig. 1. The fragmentation AUC gap (intact – row-level) is a monotone function of the sequence-essentiality diagnostic. (a) vs. Δ_{seq} (seq – single-step LSTM), with the synthetic controlled backbone and the gap = x line (the *falsified* bound). (b) vs. the order-isolating Δ_{traj} (seq – shuffled-window LSTM): tighter ($\rho_s=0.95$) and pulls the persistent `brate_med` deviator onto the origin. Order-free targets (CQI/MCS/buffer/throughput) cluster at gap ≈ 0 ; BLER targets rise with the diagnostic. Slopes < 1 in (b) reflect $\Delta_{\text{traj}} \geq \text{gap}$ (it over-predicts; Sec. V).

(max 0.025, an order below the LSTM’s 0.16) $\rightarrow$ the gap is sequence-specific. LSTM and GRU both learn the BLER trajectory (intact ≈ 0.90) and show the gap tracking Δ_{seq} . Our small Transformer has no positional encoding, so self-attention with mean-pooling is permutation-invariant—it is *order-blind by construction* (intact BLER 0.69 $\approx$ MLP 0.66) and acts as a second no-sequence sanity: both order-blind models show a negligible BLER gap (MLP +0.0001, Transformer +0.006), an order of magnitude below the order-using LSTM/GRU (+0.16/+0.15). A positional-encoded Transformer is untested (attention’s permutation-invariance impeding order modeling is a known limitation).

5.4 Twinning negative control + hunt. On Open RAN Commercial Traffic Twinning [14] (entity = UE; real Madrid LTE traffic twinned via Colosseum), the mechanism replicates (audit intact 1.0 vs. row ~ 0 , 6858 (run,UE) groups, 31M rows) but the AUC impact does not (next-step CQI/MCS gap ~ 0). A hunt for a *within-Twinning* sequence-essential positive (drop-event targets) found targets that are Δ_{traj} -positive (0.07–0.13) yet show no gap (+0.002): row-level windowing keeps each window sorted-ascending (only gappy), so it **preserves order and destroys only consecutiveness** (the row AUC tracks the ordered seqC, not the shuffled baseline), and a drop-event needs order but tolerates gaps. So $\Delta_{\text{traj}} \geq \text{gap}$ empirically; it conflates order with consecutiveness. Even Twinning’s sequence-essential targets are fragmentation-robust.

VI. A CORRECT PARTITIONING PROTOCOL

(1) Partition by natural *entity/run* (each holds its intact stream). (2) To synthesise heterogeneity, use **run-level Dirichlet** (whole runs, skewed assignment), never row-level. (3) Pre-screen with Δ_{traj} , then *confirm with the run-level-vs-row-level control*: a $\Delta_{\text{traj}} \approx 0$ benchmark is safe; a large Δ_{traj}

TABLE IV
SPOT-AUDIT BY STATED PARTITION RECIPE.

benchmark	stated partition	susceptibility
Beijing air-quality FL [15]	entity (per station), windows post-partition	safe
Mobile-net traffic FL [10]	entity (per BS)	safe
5G BS traffic [11]	entity (per BS)	safe
NIID-Bench / ProFed [2], [16]	Dirichlet over rows (image/tabular)	convention’s home; unsafe if transplanted to a pooled TS
ColO-RAN fl_v7 (this work)	Dirichlet over rows of a pooled series	at-risk (manufactured the inverted- α artifact)

flags a benchmark to investigate, but Δ_{traj} can over-predict (it destroys the last-step/order signal that END-aligned fragmentation preserves), so the partition control is the decisive test.

A spot-audit of representative FL-TS benchmarks (Table IV; we classify the stated recipe—a definitive verdict needs each paper’s code) shows well-designed *domain* benchmarks predominantly partition by *entity* (safe); the risk is transplanting the generic image/tabular Dirichlet-over-rows convention onto a pooled time series. The partition-then-window *order is rarely reported*, so susceptibility often cannot be determined from the text—its finding motivating our reporting recommendation. We do not assert any prior benchmark is flawed.

VII. LIMITATIONS AND THREATS TO VALIDITY

The $\Delta_{\text{seq}}/\Delta_{\text{traj}}$ law is an *empirical* monotone diagnostic, not a proven bound (synthetic super-linearity vs. real sub-

linearity shows the constant is data-dependent). Δ_{traj} *overpredicts* for order-but-gap-robust targets (Sec. V); it is a cheap screen, and the run-level-vs-row-level control is ground truth. Architecture coverage is LSTM + GRU (order-using) plus a mean-pool MLP and a no-positional-encoding Transformer (both order-blind sanitities); a positional-encoded Transformer is untested. The Twinning AUC-impact null is a 1-seed smoke; CoLO-RAN gaps use 5-seed paired bootstrap ($n=5$). `ul_sinr` was dropped pre-results (degenerate: median at a mass point $\rightarrow$ undefined AUC). Partition client counts differ by mode (iid = 7 BS; Dirichlet = 8), immaterial to the global-test gap.

VIII. CONCLUSION

Row-level Dirichlet partitioning—a near-universal FL-benchmark convention—can manufacture spurious findings in time-series FL by fragmenting per-client windows. The pitfall is real but *task-conditional*: it bites sequence-essential tasks (large Δ_{traj}) and spares order-free ones, which is why it has gone unnoticed. We provide the audit metric, the Δ_{traj} diagnostic, the mechanism, and a corrected protocol. We urge time-series FL benchmarks to partition by entity/run (or run-level Dirichlet) and to report Δ_{traj} and the partition-then-window order.

REPRODUCIBILITY

Pre-registration, results, theory, and scripts (synthetic; CoLO-RAN sweep; Δ_{traj} addendum; architecture invariance; Twinning audit + AUC-impact + hunt; `window_cache` with a bit-exact test) are in the project repository. All runs: local RTX 4060 Ti (memory-caged) + a V100 CoLO-RAN sweep.

REFERENCES

- [1] T.-M. H. Hsu, H. Qi, and M. Brown, “Measuring the effects of non-identical data distribution for federated visual classification,” in *NeurIPS Workshop on Federated Learning*, 2019, arXiv:1909.06335.
- [2] Q. Li, Y. Diao, Q. Chen, and B. He, “NIID-Bench: Federated learning on non-IID data silos: An experimental study,” in *ICDE*, 2022, arXiv:2102.02079.
- [3] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, “Flower: A friendly federated learning research framework,” 2020, arXiv:2007.14390.
- [4] M. Polese, L. Bonati, S. D’Oro, S. Basagni, and T. Melodia, “CoLO-RAN: Developing machine learning-based xApps for Open RAN closed-loop control on programmable experimental platforms,” *IEEE Transactions on Mobile Computing*, 2022.
- [5] D. M. Jimenez-Gutierrez, M. Hassanzadeh, A. Anagnostopoulos, I. Chatzigiannakis, and A. Vitaletti, “A thorough assessment of the non-IID data impact in federated learning,” 2025, arXiv:2503.17070.
- [6] S. Albelali and M. Ahmed, “Hidden leaks in time series forecasting: How data leakage affects LSTM evaluation across configurations and validation strategies,” 2025, arXiv:2512.06932.
- [7] I. Misra, C. L. Zitnick, and M. Hebert, “Shuffle and learn: Unsupervised learning using temporal order verification,” in *ECCV*, 2016.
- [8] S. Grover, A. Jalali, and A. Etemad, “Segment, shuffle, and stitch: A simple layer for improving time-series representations,” in *NeurIPS*, 2024, arXiv:2405.20082.
- [9] A. Abedi and S. S. Khan, “FedSL: Federated split learning on distributed sequential data in recurrent neural networks,” 2020, arXiv:2011.03180.
- [10] N. Pavlidis, V. Perifanis, S. F. Yilmaz, F. Wilhelmi, M. Miozzo, P. S. Efraimidis, R.-A. Koutsiamanis, P. Mulinka, and P. Dini, “Federated learning in mobile networks: A comprehensive case study on traffic forecasting,” 2024, arXiv:2412.04081.
- [11] V. Perifanis, N. Pavlidis, R.-A. Koutsiamanis, and P. S. Efraimidis, “Federated learning for 5g base station traffic forecasting,” *Computer Networks*, vol. 235, p. 109950, 2023, arXiv:2211.15220; code: github.com/vperifan/Federated-Time-Series-Forecasting.
- [12] Y. Dutta, S. Chatterjee, S. Chakraborty, and B. Palit, “FedAvg/FedProx/FedBN benchmark on five throughput-prediction datasets,” 2025, arXiv:2508.08479.
- [13] L. Bonati, M. Polese, S. D’Oro, S. Basagni, and T. Melodia, “OpenRAN Gym: An open toolbox for data collection and experimentation with AI in O-RAN,” in *Proc. ACM WiNTECH*, 2022, arXiv:2202.10318.
- [14] L. Bonati, R. Shirkhani, C. Fiandrino, S. Maxenti, S. D’Oro, M. Polese, and T. Melodia, “Twinning commercial network traces on experimental open RAN platforms,” in *Proc. ACM WiNTECH*, 2024, arXiv:2409.16217.
- [15] K. Hallak and O. Kem, “Benchmarking catastrophic forgetting mitigation methods in federated time series forecasting,” in *IEEE Conf. on Federated Learning Technologies and Applications (FLTA)*, 2025, arXiv:2510.21491.
- [16] D. Domini, G. Aguzzi, and M. Viroli, “ProFed: a benchmark for proximity-based non-IID federated learning,” 2025, arXiv:2503.20618.